Experiment 09

Deepa Garg **Semester 05**

2401029 T1 **ASD Lab**

Aim

Expt 9: To provision cloud infrastructure using Terraform.

Lab Outcomes

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory

Identity and Access Management (IAM)

AWS Identity and Access Management (IAM) allows an AWS account owner to create and manage individual users and control what each user is permitted to do. Instead of sharing root account credentials, a dedicated IAM user (such as Experiment7 in this experiment) can be created and given only the permissions required for a specific task, following the principle of least privilege. Permissions are granted using policies, such as the AWS-managed AmazonEC2FullAccess and AmazonVPCFullAccess policies, which allow the user to create and manage EC2 instances and their associated networking resources.

Programmatic Access and Access Keys

An IAM user's console password (used to sign in through a browser) is separate from the access keys used for programmatic access. An Access Key ID and Secret Access Key pair allows tools such as the AWS CLI, SDKs, or Terraform to authenticate API requests on behalf of the user. AWS shows the secret access key only once at creation time, so it must be copied or downloaded immediately, and access-key best practice recommends never storing it in plain text or in source code.

AWS Command Line Interface (AWS CLI)

The AWS CLI is a unified command-line tool that allows a user to interact with AWS services directly from a terminal. After installation, running aws configure prompts for the Access Key ID, Secret Access Key, default region (ap-south-1 in this experiment), and default output format (json), storing them so that subsequent AWS CLI and Terraform commands can authenticate automatically.

Terraform and Infrastructure as Code (IaC)

Terraform is an open-source Infrastructure as Code (IaC) tool that allows cloud resources to be defined declaratively in configuration files and provisioned automatically. Rather than manually clicking through the AWS console, the desired infrastructure is described in code, which can be version-controlled, reviewed, and reused, making deployments consistent and repeatable. All .tf files placed in

the same folder are treated as a single Terraform configuration; the filename does not determine what gets executed.

main.tf and outputs.tf

main.tf defines the required provider (the hashicorp/aws provider, the AWS region) and the resources to be created, such as the aws_instance resource representing an EC2 virtual machine. outputs.tf defines values, such as the instance's ID, instance type, and public IP address, that Terraform should display once the resources have been created, making it easy to retrieve key details without checking the AWS console.

The Terraform Workflow

- terraform init — downloads and installs the required provider plugins (the HashiCorp AWS provider) and initializes the working directory.
- terraform validate — checks that the configuration files are syntactically valid and internally consistent.
- terraform plan — compares the desired configuration against the current state and shows exactly which resources will be added, changed, or destroyed, without making any changes.
- terraform apply — executes the plan, provisioning the actual AWS resources, and asks for explicit confirmation (yes) before proceeding.
- terraform output — displays the values defined in outputs.tf for the currently applied infrastructure.
- terraform destroy — tears down all resources that Terraform created, which is essential for withdrawing access and avoiding unnecessary AWS charges once the experiment is complete.

Implementation

Step 1: Create IAM User 'Experiment7' and Attach Permissions

On the Set permissions screen, Attach policies directly is selected and the AmazonVPCFullAccess policy (along with AmazonEC2FullAccess) is attached to the new user.

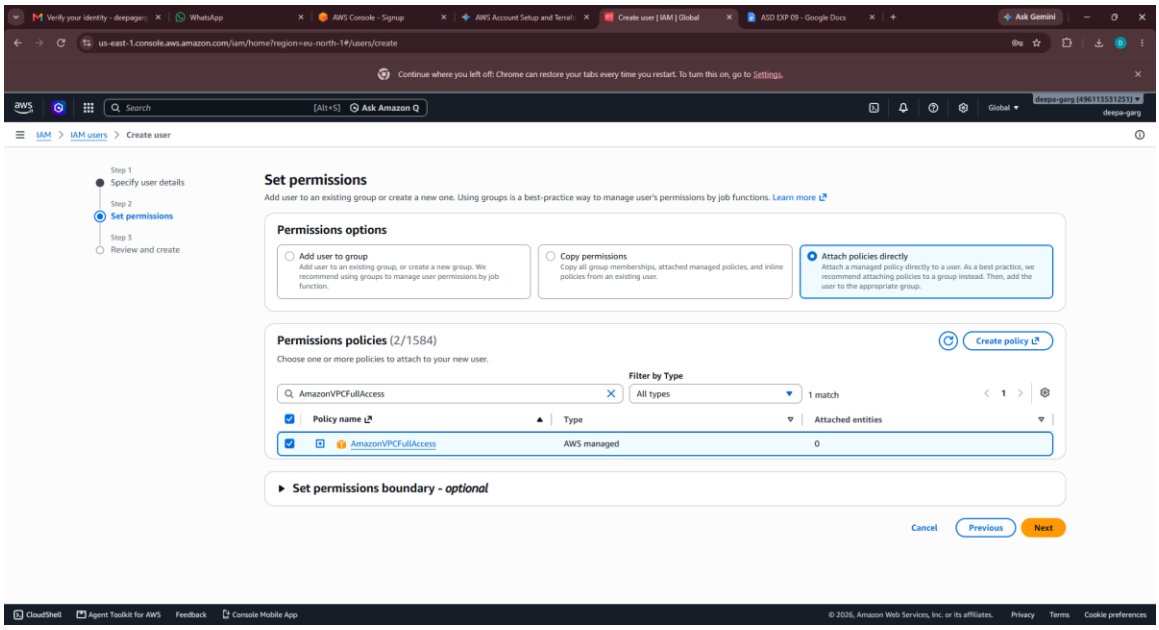


Fig 1: Attaching AmazonVPCFullAccess and AmazonEC2FullAccess policies directly to the new IAM user

The Review and create screen summarizes the new user's name (Experiment7) and the two permissions policies selected before the user is created.

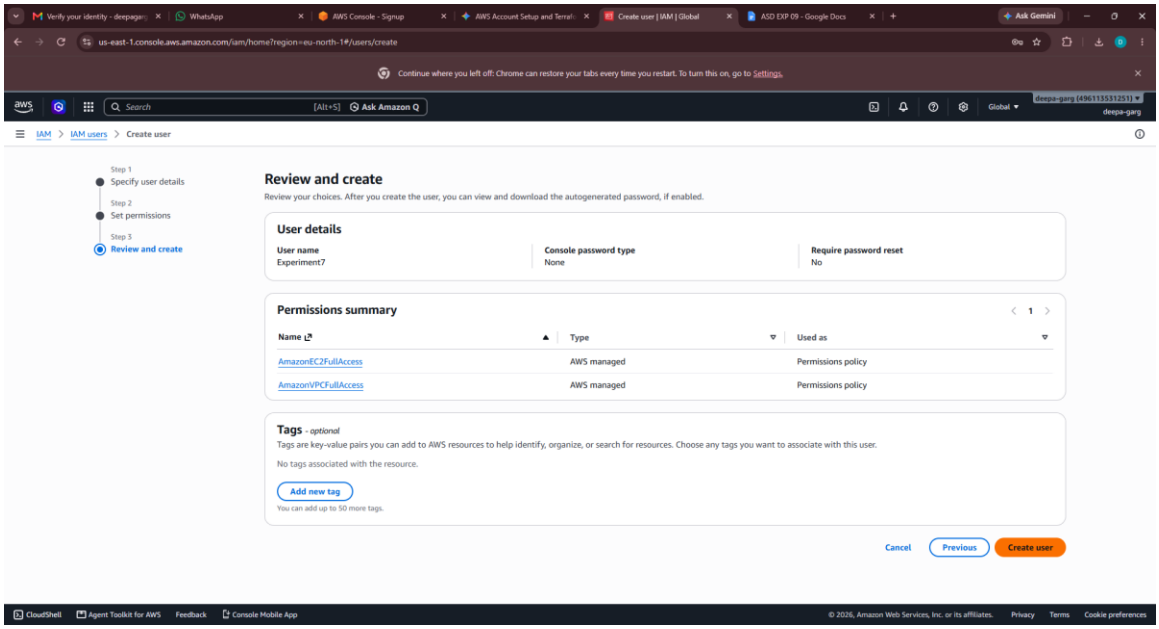


Fig 2: Reviewing user details and attached permissions before creating the IAM user

Once created, IAM confirms that the user was created successfully, and the Permissions tab shows both AmazonEC2FullAccess and AmazonVPCFullAccess attached directly to Experiment7.

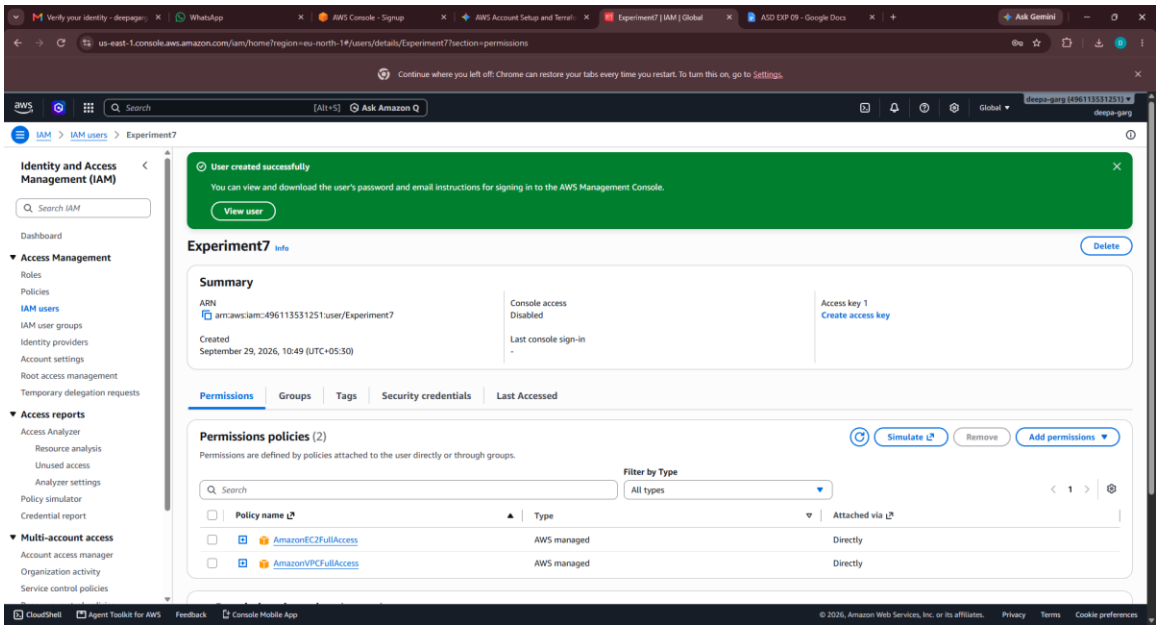


Fig 3: Confirmation that IAM user 'Experiment7' was created successfully with both policies attached

Step 2: Review Security Credentials and Locate Access Key Options

The Security credentials tab for Experiment7 shows the console sign-in link and Multi-Factor Authentication (MFA) settings for the user.

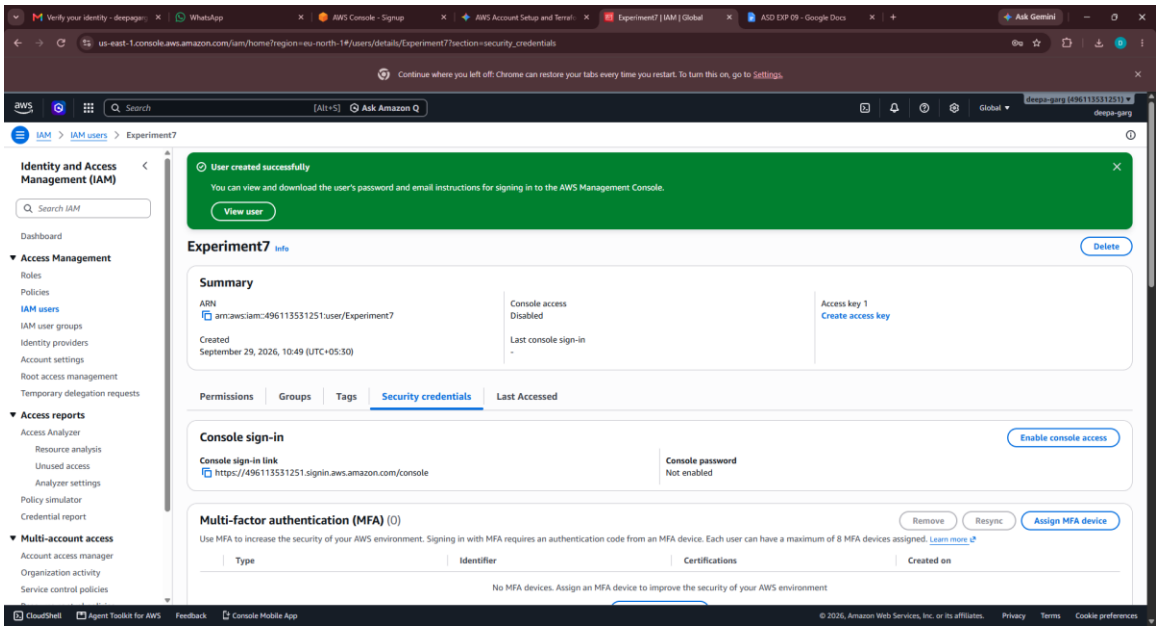


Fig 4: Security credentials tab for Experiment7, showing console sign-in and MFA settings

Scrolling further down the same tab reveals the Access keys (0) section, from which a new access key can be created for programmatic access.

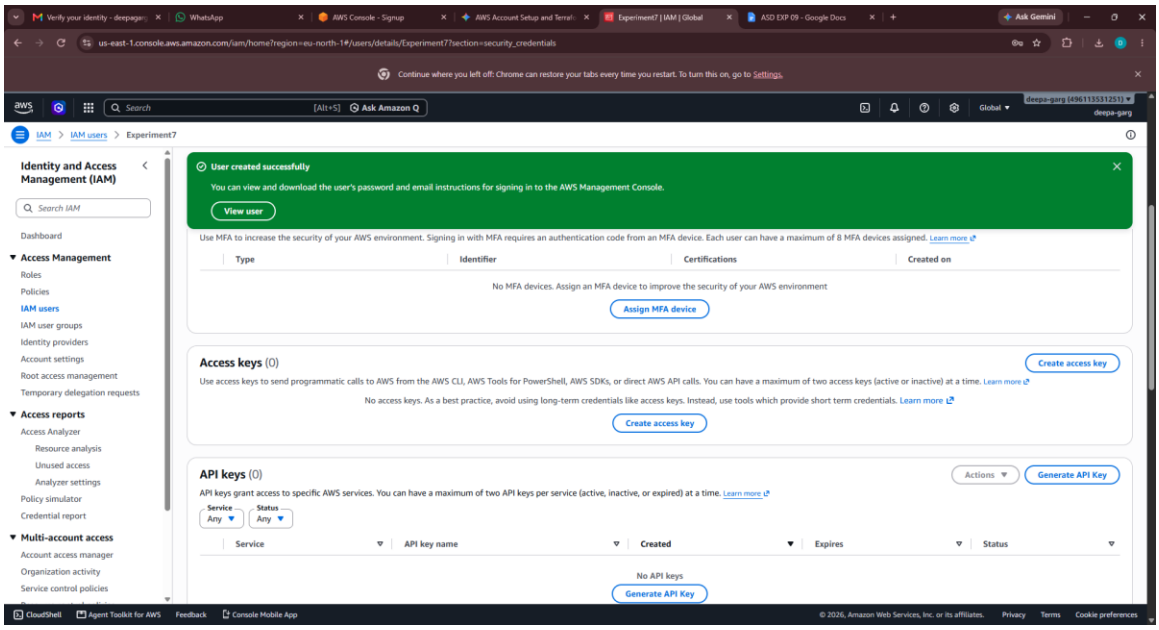


Fig 5: Locating the Access keys section to create a new access key for Experiment7

Step 3: Generate and Retrieve IAM Access Keys

Command Line Interface (CLI) is selected as the use case, indicating that the access key will be used to authenticate the AWS CLI.

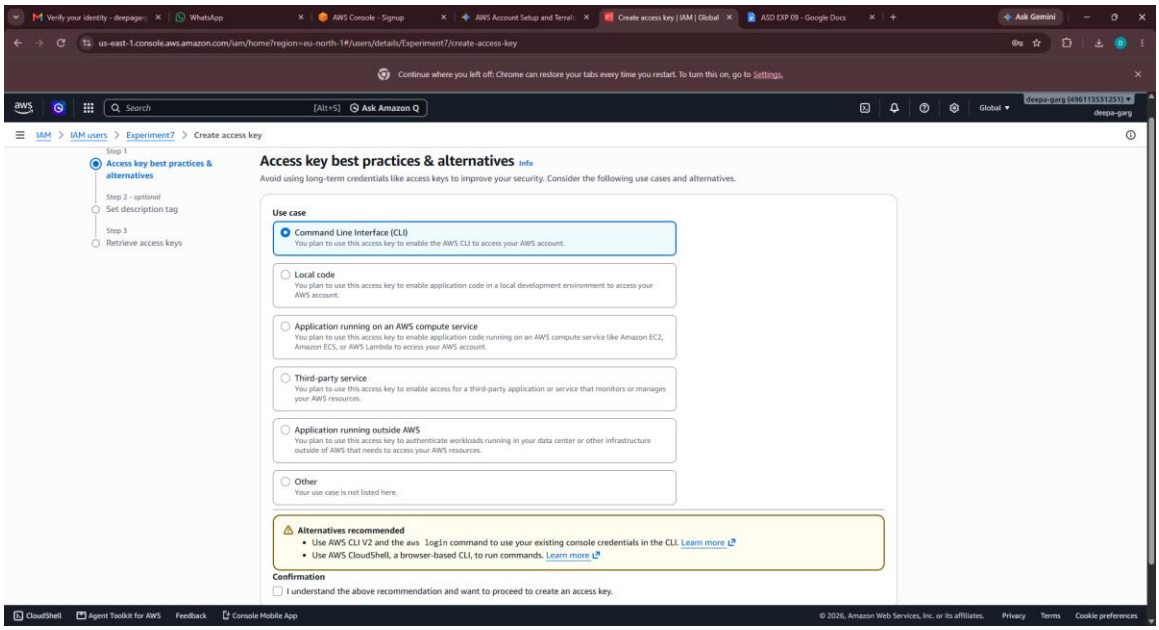


Fig 6: Selecting Command Line Interface (CLI) as the use case for the new access key

The generated Access Key ID and Secret Access Key are displayed; AWS warns that the secret key can only be viewed or downloaded at this point.

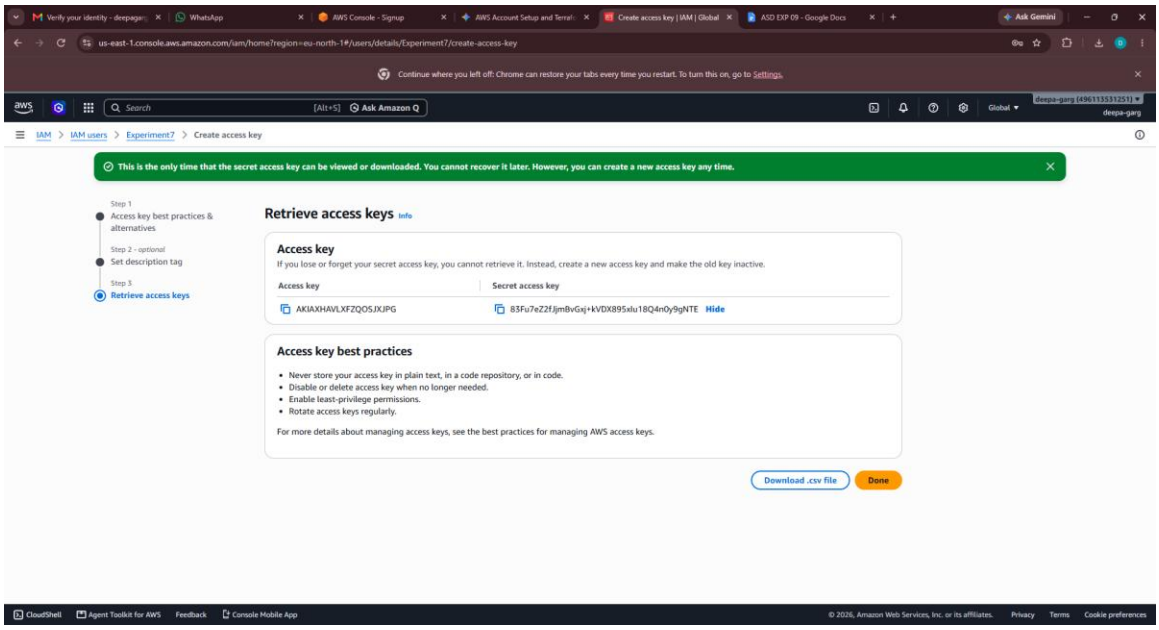


Fig 7: Retrieving the generated Access Key ID and Secret Access Key

The same screen is shown with the secret access key hidden, confirming the show/hide toggle behaviour used to protect the credential from casual viewing.

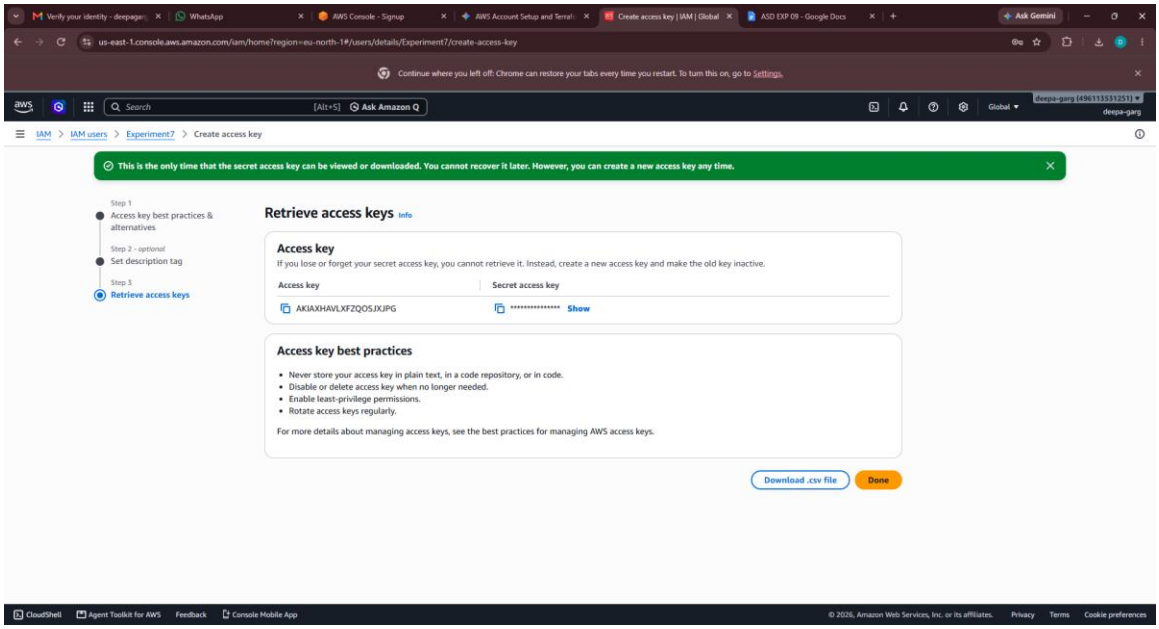


Fig 8: Access key retrieval screen with the secret access key hidden

Returning to the Experiment7 summary confirms that Access Key 1 is now Active and ready to be used to configure the AWS CLI.

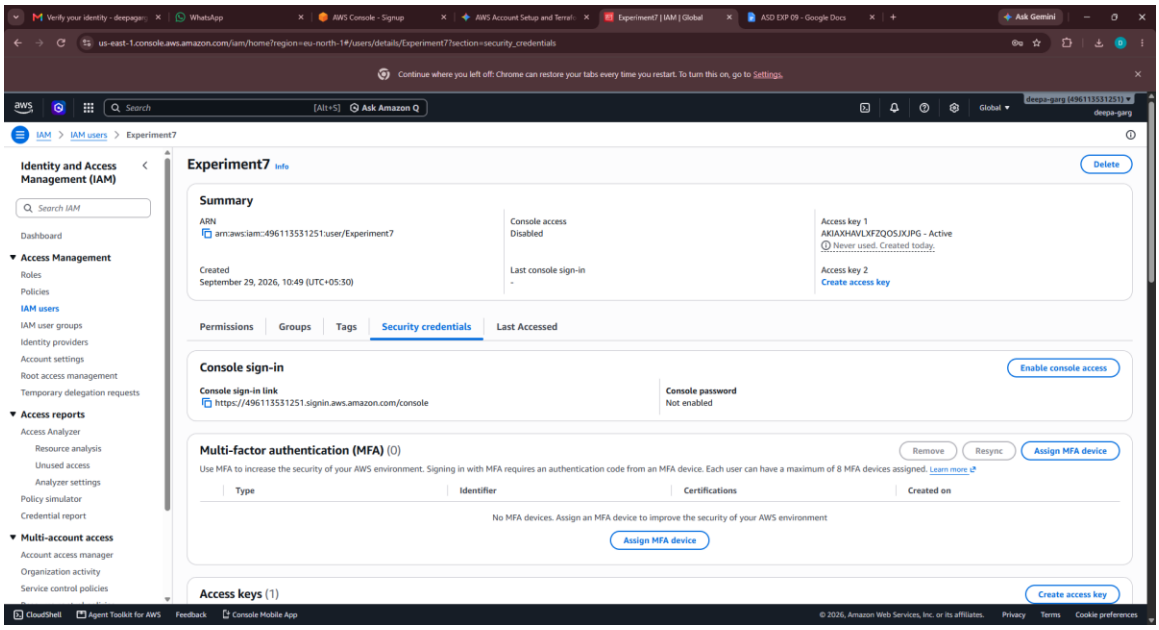


Fig 9: Experiment7 user summary confirming that Access Key 1 is active

Step 4: Install AWS CLI on Windows via PowerShell

The AWS CLI v2 installer script is executed directly from PowerShell (irm https://awscli.amazonaws.com/v2/install.ps1 | iex), which downloads and installs the AWS CLI.

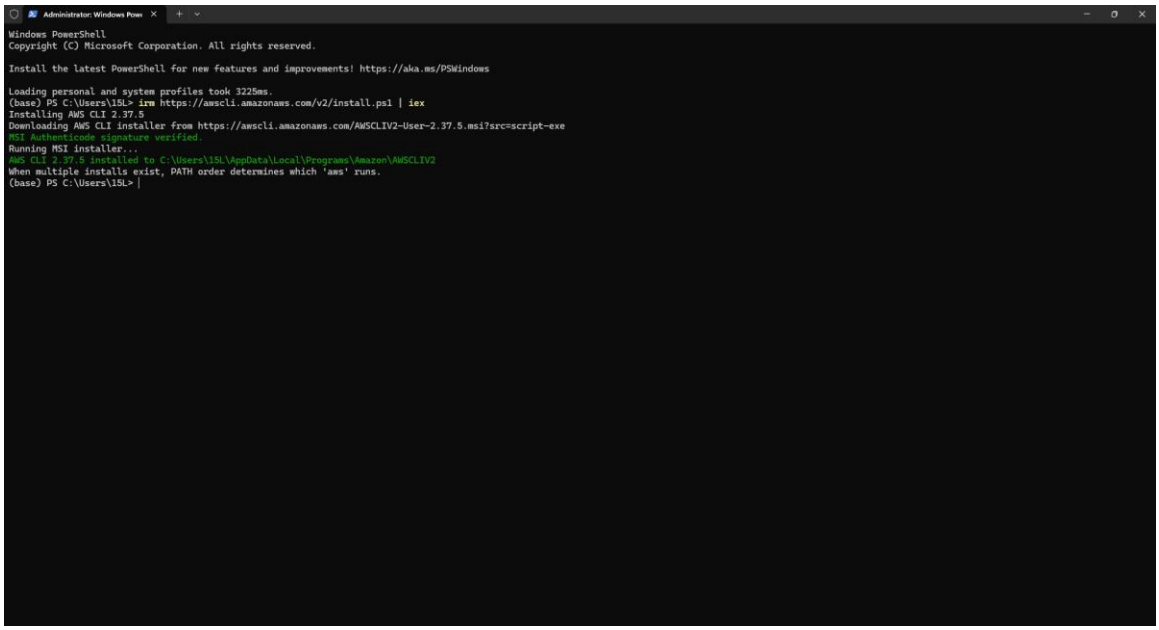


Fig 10: Installing AWS CLI v2 on Windows using a PowerShell script

Step 5: Configure AWS Credentials using AWS CLI

After resolving an initial PATH issue by calling the AWS CLI executable directly, aws configure is run and the Access Key ID, Secret Access Key, default region (ap-south-1), and default output format (json) are entered, after which aws sts get-caller-identity style checks confirm the credentials are recognised.

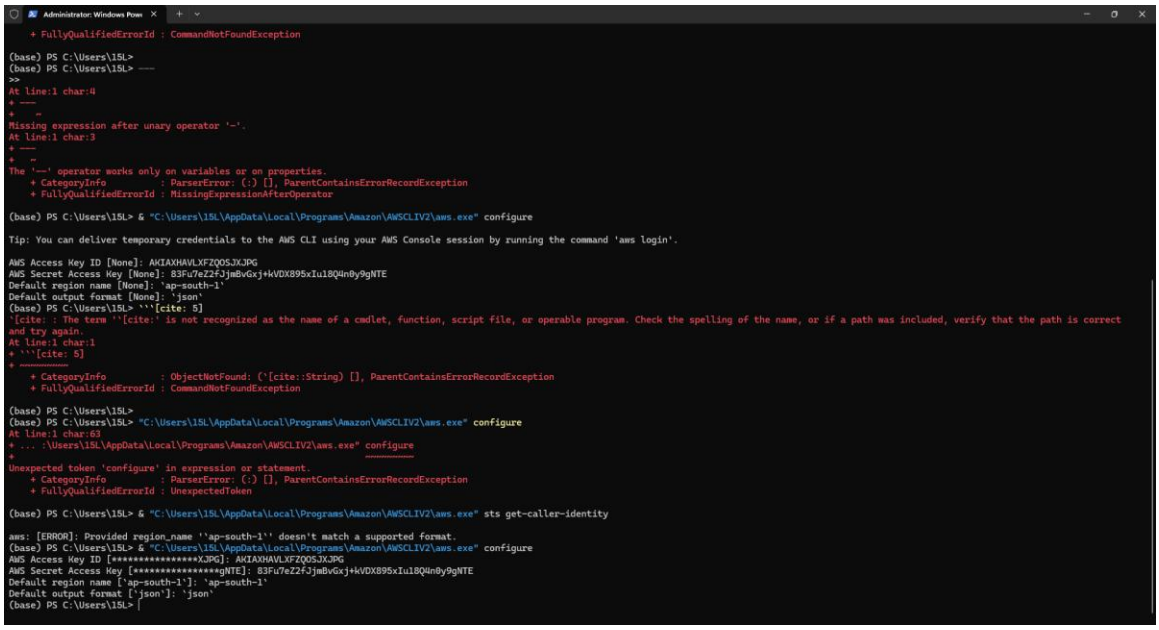


Fig 11: Running 'aws configure' and entering the Access Key, Secret Key, region, and output format

Step 6: Create Terraform Configuration Files (main.tf & outputs.tf)

main.tf declares the required hashicorp/aws provider, sets the AWS region, and defines the aws_instance resource (student_vm) with its AMI ID, instance type (t3.micro), and a Name tag.

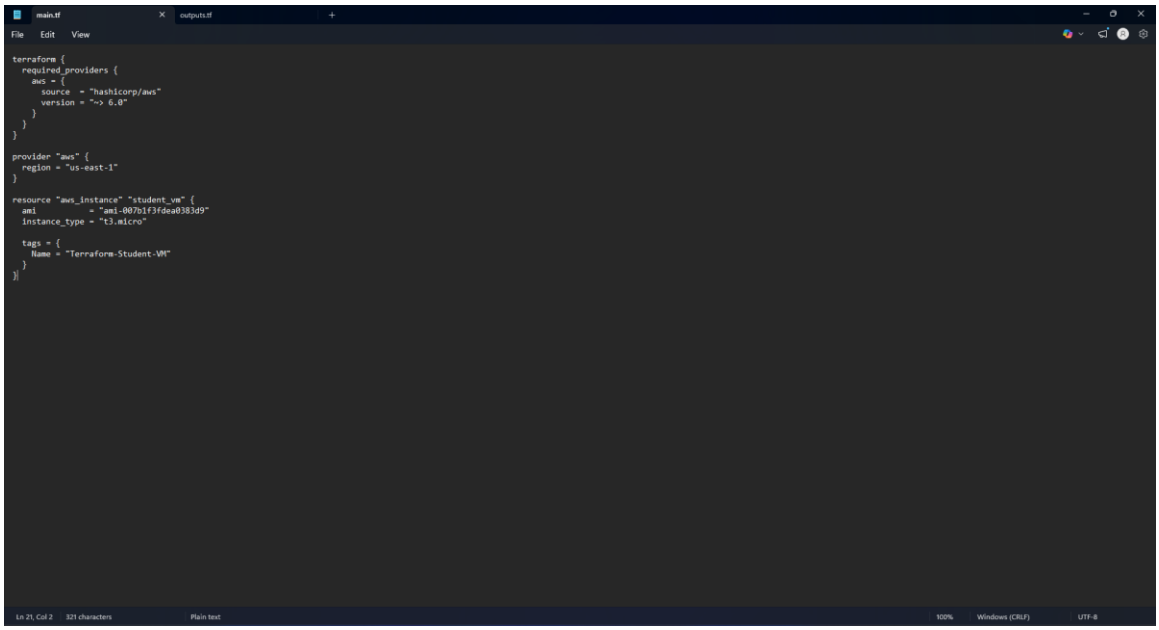


Fig 12: main.tf defining the AWS provider and the aws_instance resource

outputs.tf defines three outputs — instance_id, public_ip, and instance_type — so that key details of the created instance are displayed automatically after apply.

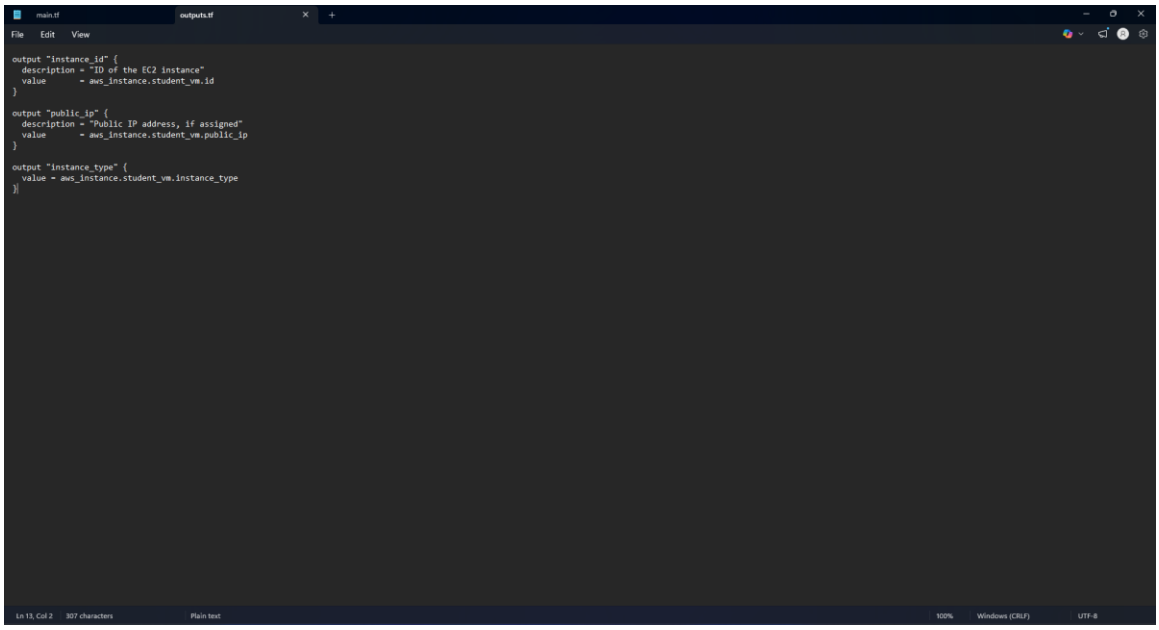


Fig 13: outputs.tf defining the instance_id, public_ip, and instance_type outputs

Step 7: Initialize, Validate, and Plan the Terraform Configuration

terraform init downloads and installs the hashicorp/aws provider plugin and initializes the working directory; terraform validate then confirms the configuration is syntactically valid ('Success! The configuration is valid.'), after which terraform plan begins generating the execution plan.

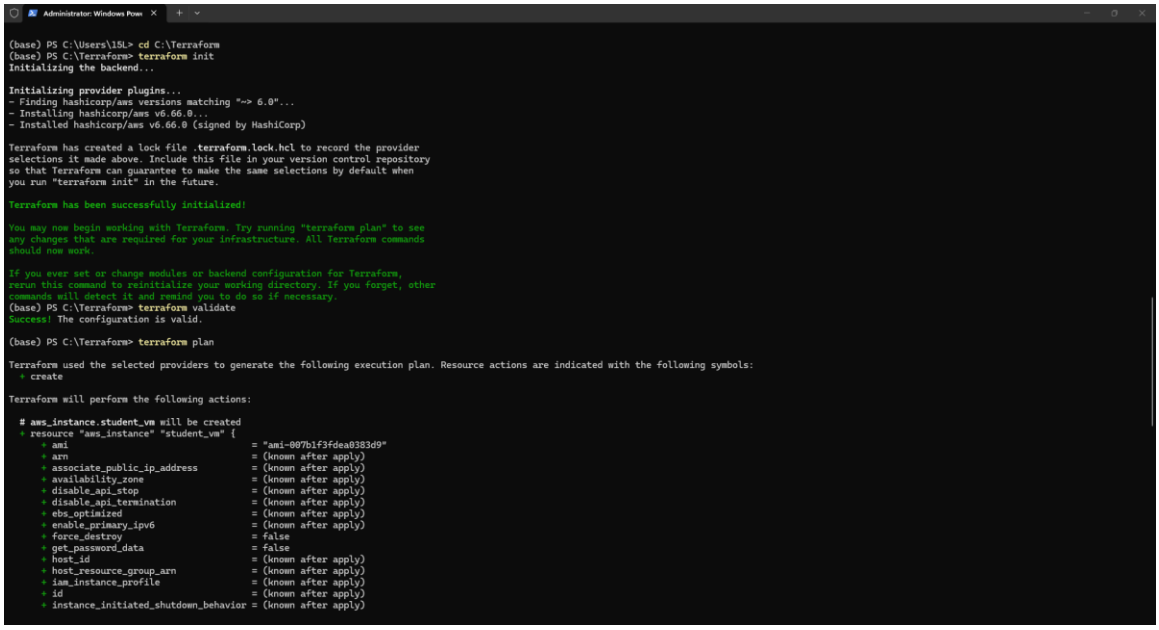


Fig 14: Running terraform init and terraform validate, followed by the start of terraform plan

Step 8: Review the Terraform Execution Plan

terraform apply is run, and Terraform begins listing every attribute of the aws_instance resource that will be created, most of which are shown as (known after apply).

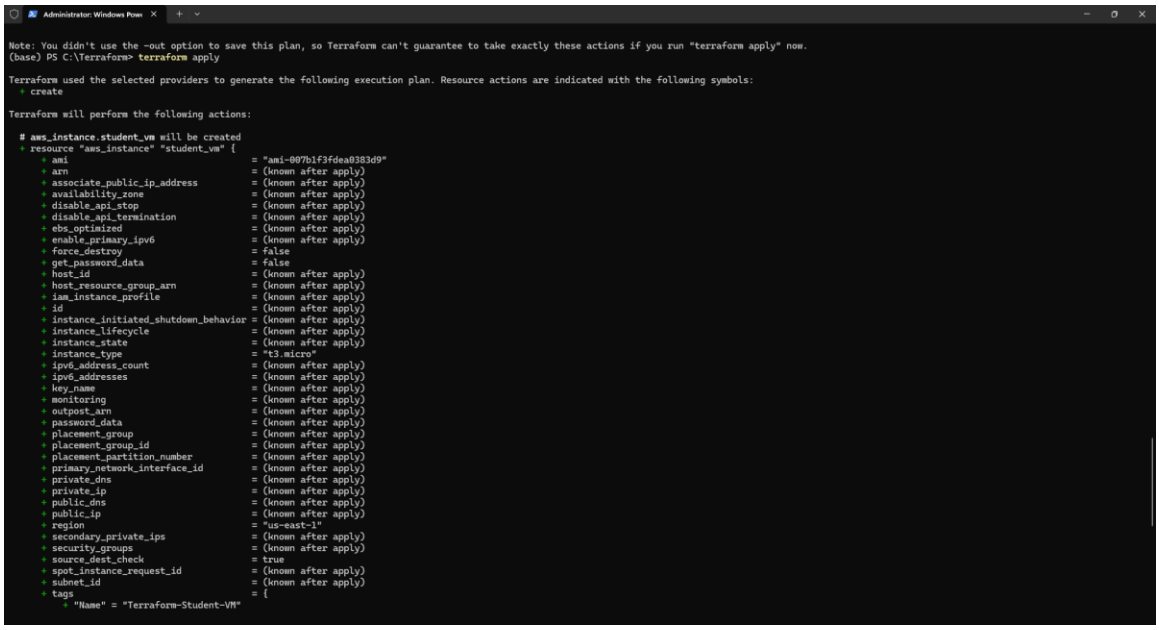


Fig 15: terraform apply invoked; execution plan listing the attributes of the aws_instance to be created

The plan output continues, listing the remaining instance attributes and nested blocks such as ebs_block_device and network_interface.

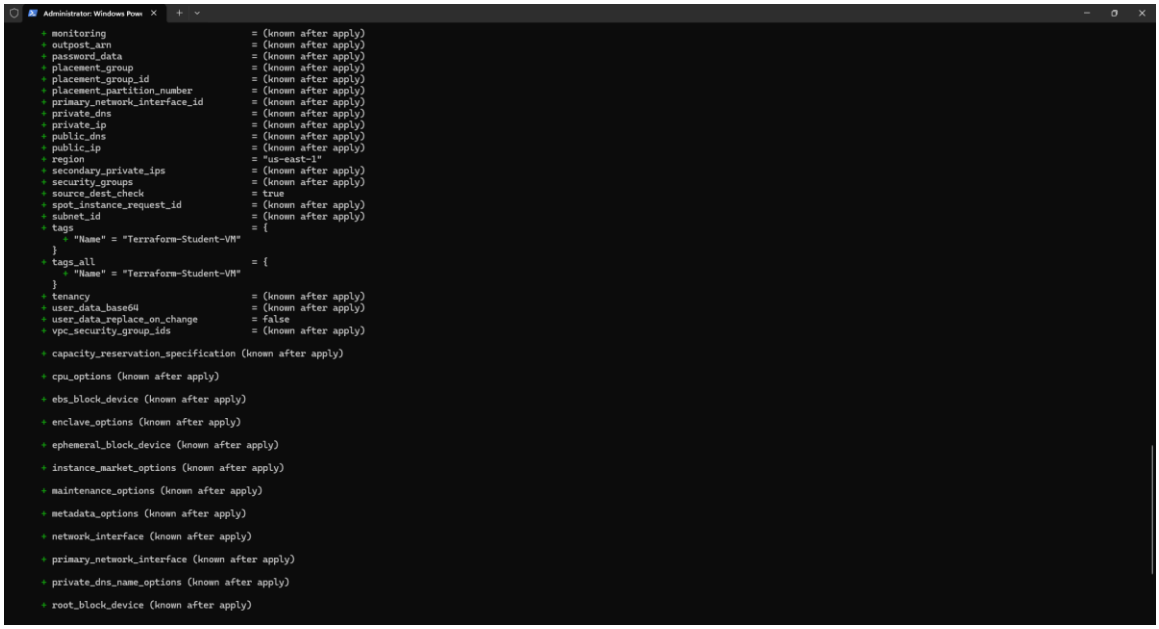


Fig 16: Execution plan continued, showing the remaining instance attributes

The plan concludes with a summary (Plan: 1 to add, 0 to change, 0 to destroy) and the changes to outputs, and 'yes' is entered to approve the actions.

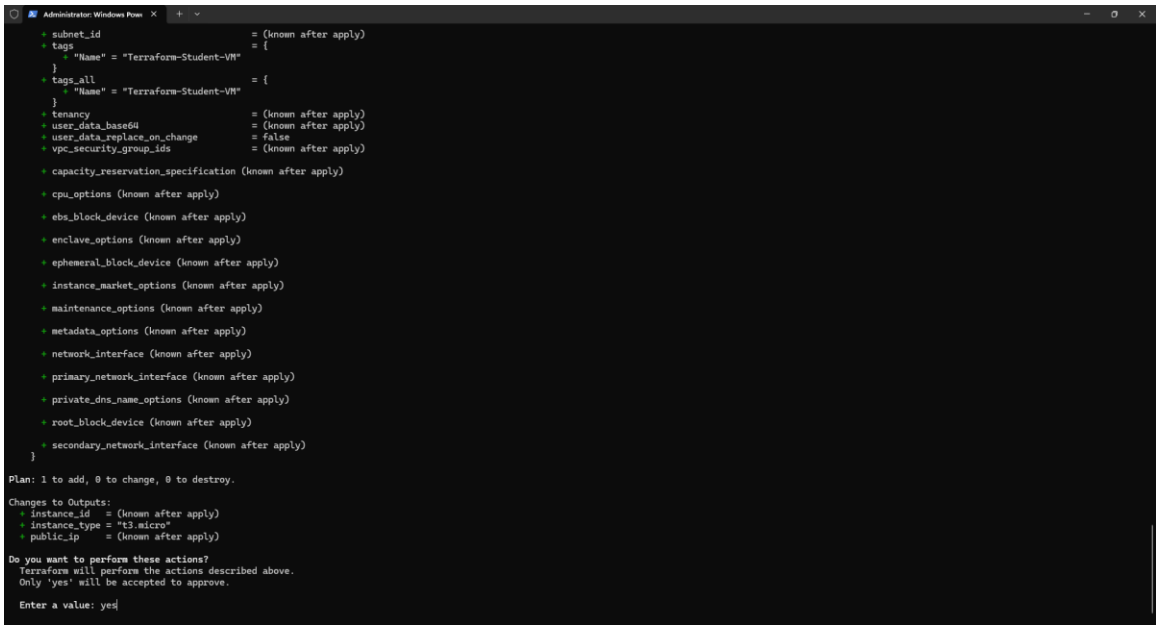


Fig 17: End of the execution plan and approving the apply by entering 'yes'

Step 9: Apply the Configuration and Verify Outputs

Terraform reports that the aws_instance.student_vm resource was created, and prints 'Apply complete! Resources: 1 added, 0 changed, 0 destroyed.' along with the instance_id, instance_type, and public_ip outputs.

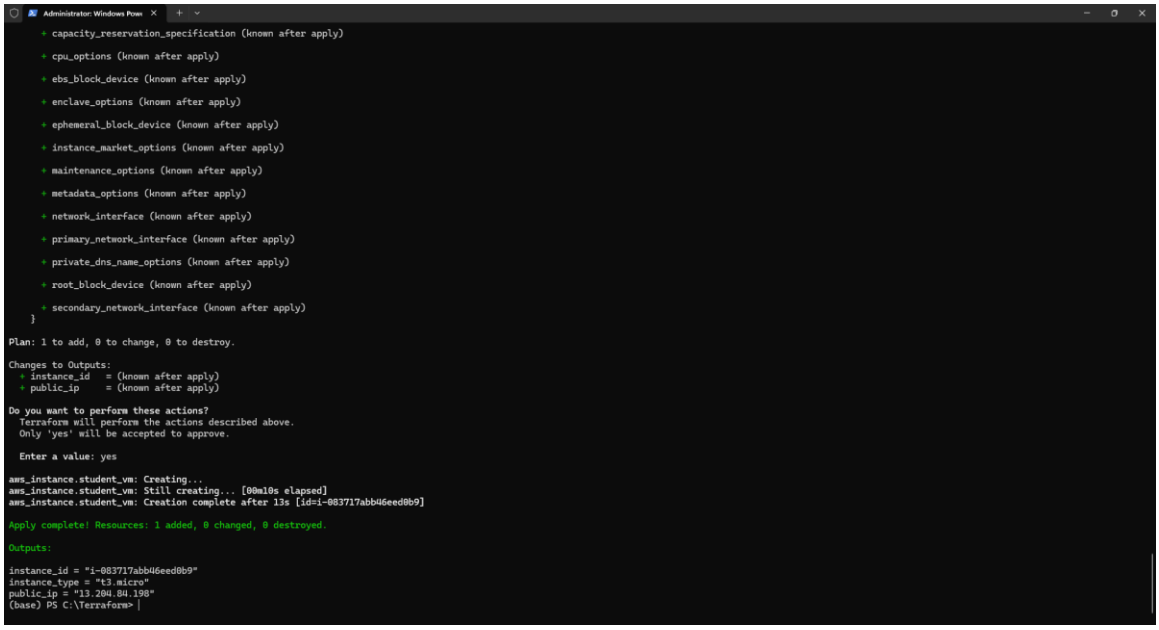


Fig 18: 'Apply complete!' with the instance_id, instance_type, and public_ip outputs

Running terraform output afterwards redisplays the same output values, confirming they persist in the Terraform state.

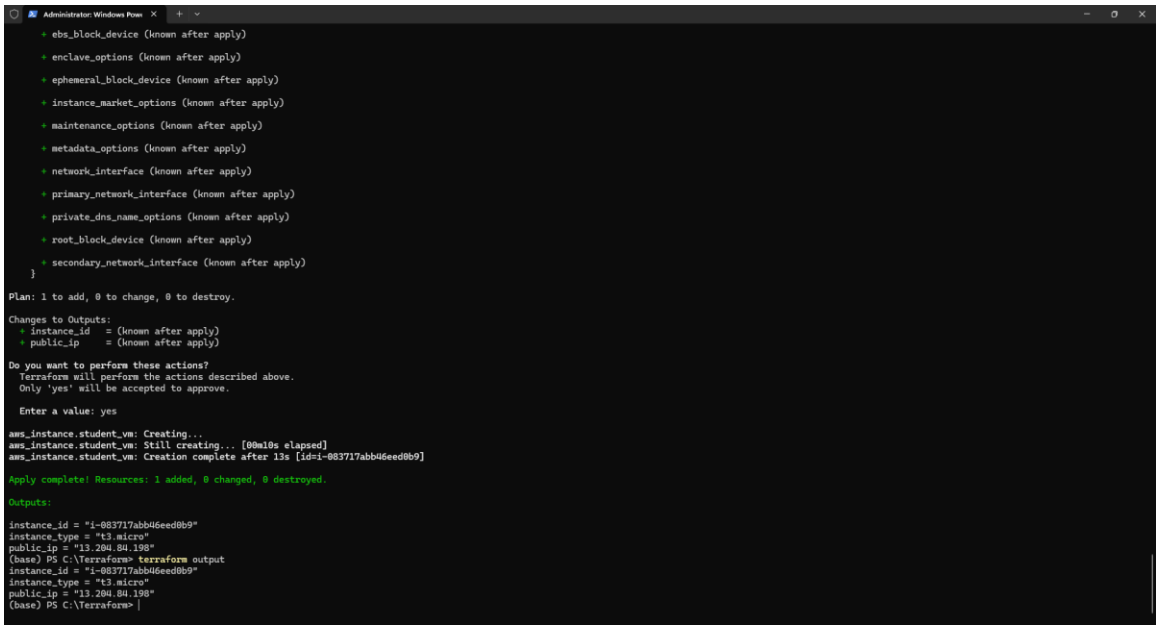


Fig 19: Running 'terraform output' to redisplay the instance_id, instance_type, and public_ip

Step 10: Verify the Deployed EC2 Instance in the AWS Console

The AWS EC2 console (Asia Pacific – Mumbai region) confirms that the instance Terraform-Stu... is Running, of type t3.micro, with the same instance ID and public IPv4 address reported by Terraform.

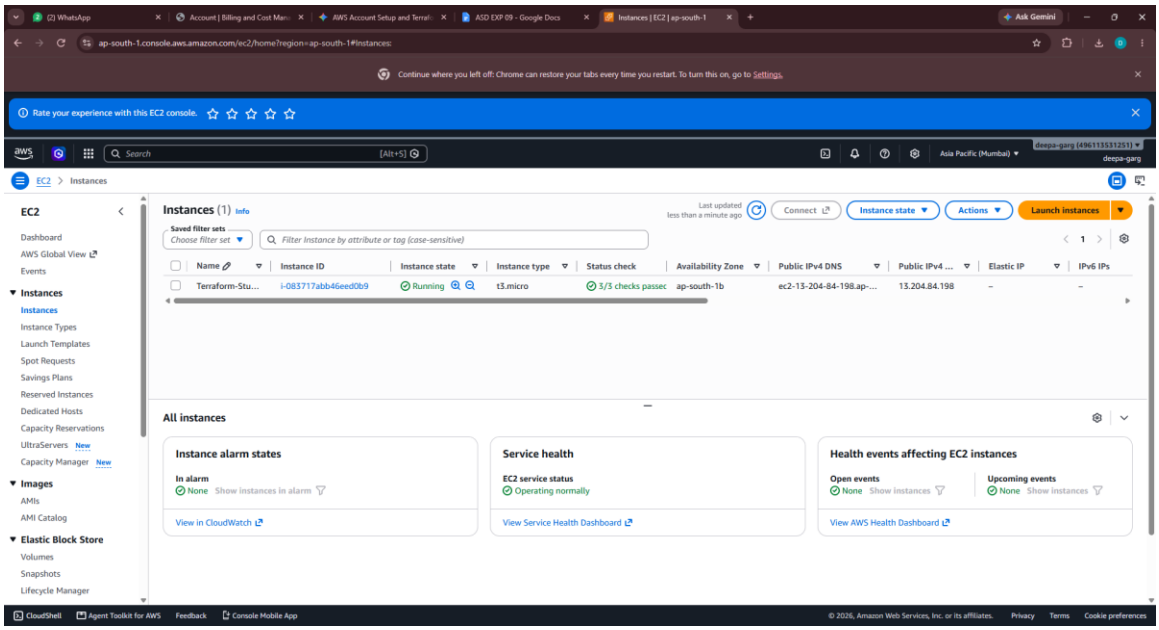


Fig 20: AWS EC2 console confirming the running 'Terraform-Student-VM' instance in ap-south-1

Step 11: Destroy the Terraform-Managed Infrastructure

terraform destroy is run; Terraform refreshes the current state and generates a destroy plan showing that aws_instance.student_vm will be destroyed, with every attribute changing from its current value to null.

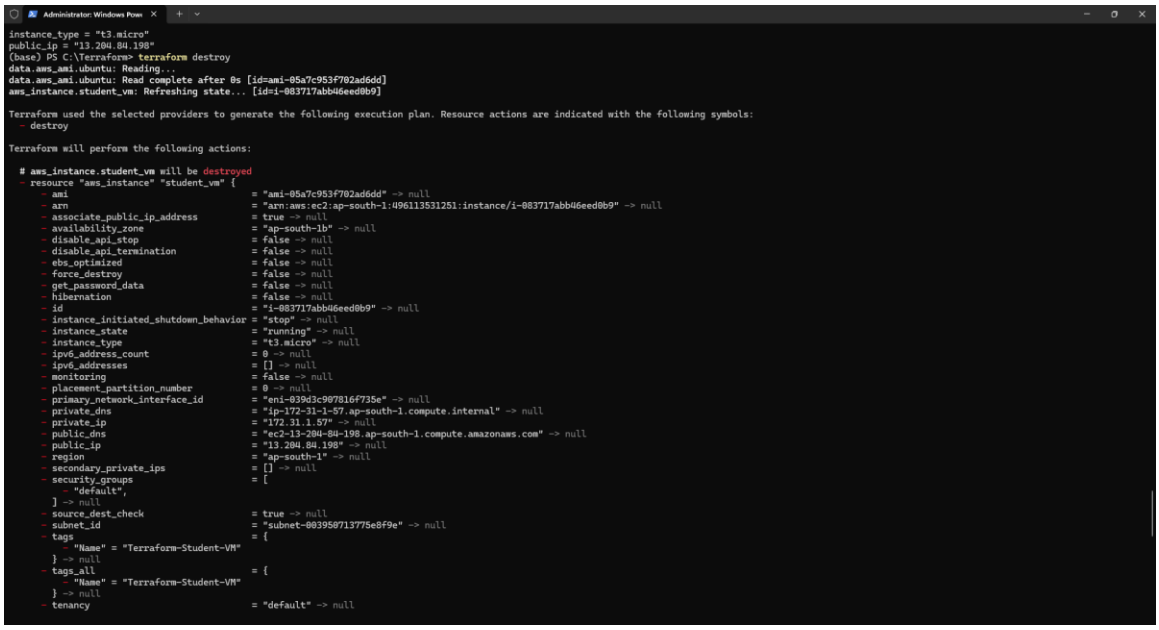


Fig 21: terraform destroy execution plan showing aws_instance.student_vm will be destroyed

After confirming with 'yes', Terraform reports 'Destroy complete! Resources: 1 destroyed.', confirming that the EC2 instance and its associated outputs have been removed.

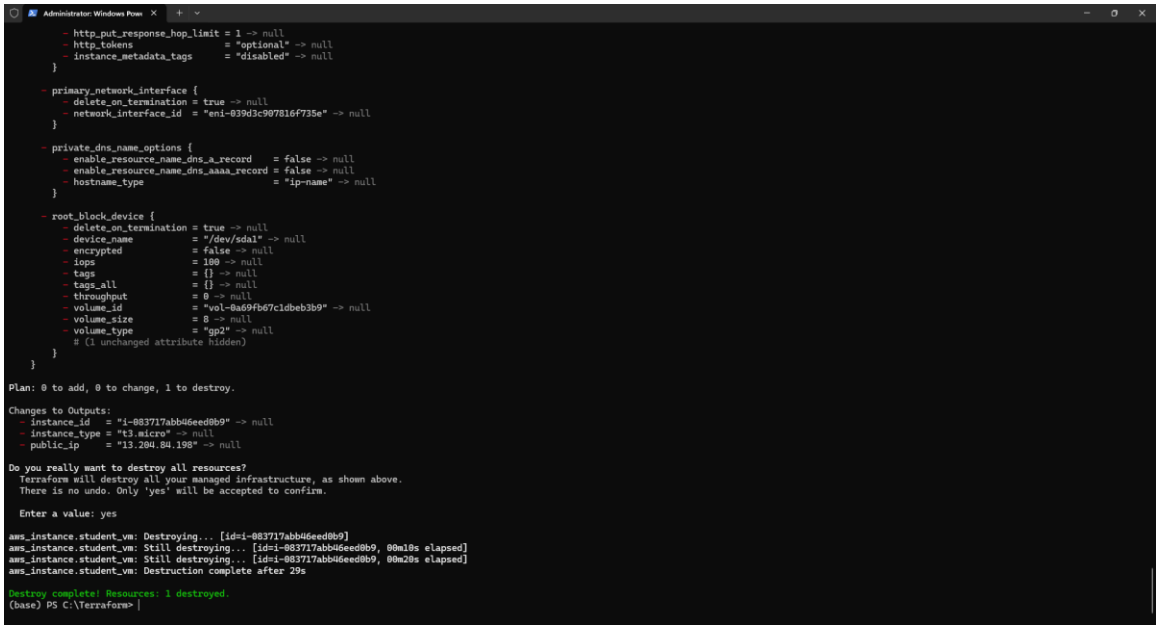


Fig 22: 'Destroy complete!' confirming the EC2 instance was successfully removed

Conclusion

Gained practical knowledge of creating and managing IAM users and policies, generating and using programmatic access keys, configuring the AWS CLI, and using Terraform to provision, verify, and destroy AWS EC2 infrastructure through Infrastructure as Code (IaC).

Terraform, combined with IAM-based access control, enables automated and consistent provisioning of cloud infrastructure, supporting secure, auditable, and efficient deployment and management of AWS resources in real-world DevOps and cloud-engineering pipelines.

The experiment demonstrates Infrastructure as Code practices together with the principle of least-privilege access via IAM, improving scalability, reliability, repeatability, security, and efficiency in cloud-based engineering.

Developed skills in creating IAM users and policies, generating and securely handling access keys, configuring the AWS CLI, writing Terraform configurations (main.tf, outputs.tf), and executing the full Terraform workflow — init, validate, plan, apply, output, and destroy — to provision and tear down AWS resources.